

Consigna del Trabajo Práctico: Simulador de Presupuestos y Control de Stock

Asignatura: Bases de Datos 2

Modalidad: Grupal (Máximo 4 integrantes)

1. Descripción del Proyecto

Los estudiantes deberán desarrollar una aplicación de consola (CLI - Command Line Interface) que simule de forma automatizada las operaciones comerciales de un negocio (control de inventario, facturación básica y aplicación de reglas de negocio como impuestos y descuentos). El foco principal radica en la correcta separación de responsabilidades y la interacción limpia entre diferentes módulos de software.

2. Estructura Obligatoria del Código

El proyecto debe presentar una arquitectura modular estricta. Está prohibido concentrar toda la lógica en un único archivo. La estructura obligatoria es la siguiente:

```
proyecto-node/  
├─ package.json  
├─ index.js  
├─ calculosMatematicos.js  
├─ baseDeDatosSimulada.js  
└─ formateoVisual.js
```

Responsabilidades por archivo:

- **package.json:** Archivo de configuración inicializado correctamente. Debe reflejar datos claros y especificar el sistema de módulos elegido.
- **index.js:** Punto de entrada de la aplicación. Se encargará exclusivamente del flujo principal, la invocación de funciones externas y la muestra de resultados finales por consola. No debe contener funciones matemáticas ni datos duros (hardcoded).
- **calculosMatematicos.js:** Módulo puramente operativo. Debe exportar funciones para sumar subtotales, calcular el Impuesto al Valor Agregado (IVA), aplicar descuentos porcentuales por volumen de compra y restar unidades del inventario.

- **baseDeDatosSimulada.js:** Contendrá una estructura de datos en memoria (un array de objetos) que represente el catálogo de productos de la empresa. Cada producto debe poseer atributos consistentes: id, nombre, precio, y stock. Debe exportar este set de datos para su consumo en otros archivos.
- **formateoVisual.js:** Encargado de la estética de la interfaz de consola. Debe exportar funciones utilitarias que devuelvan cadenas formateadas (por ejemplo, agregar el símbolo monetario, aplicar mayúsculas a títulos o generar separadores estéticos de texto). Este es un plus si quieren sumar puntos al tp.

3. Requerimientos Funcionales

Al ejecutar el comando `node index.js`, el sistema debe realizar de forma automatizada las siguientes acciones correlativas:

1. **Lectura del Inventario Inicial:** Cargar y mostrar de forma clara y prolija el listado completo de productos disponibles, importados desde el archivo de datos.
2. **Procesamiento de una Venta/Presupuesto:** Simular la selección de al menos tres artículos del catálogo. El sistema calculará el subtotal de la compra.
3. **Aplicación de Reglas de Negocio:** Invocar las funciones del módulo matemático para aplicar un descuento del 10% si el subtotal supera un umbral preestablecido, sumar el impuesto correspondiente y retornar el total neto a pagar.
4. **Actualización Dinámica de Stock:** Modificar los valores del inventario restando las unidades vendidas. El sistema finalizará imprimiendo el estado actualizado del stock en memoria para verificar el éxito de la operación.

4. Requisitos Obligatorios de Entrega (Uso de GitHub)

- **Repositorio Centralizado:** El equipo deberá inicializar un repositorio Git local y **subir el código completo a un repositorio público en GitHub**.
- **Archivo README.md:** El repositorio debe contar con un archivo de documentación inicial en la raíz que muestre el nombre de los integrantes del grupo, el estándar de módulos seleccionado y las instrucciones precisas para clonar y ejecutar el entorno de desarrollo.